03 · TypeScript Essentials

Beginner Angular learning notes

a. Definition

TypeScript is JavaScript with **types**. You tell the language what kind of value a variable holds (`string`, `number`, `boolean`...), and the editor warns you about mistakes before you even run the code.

Angular is written in TypeScript, so you will use **interfaces** (shapes of objects), **classes**, **generics** and **decorators** (like `@Component`) every day. Browsers cannot run TypeScript directly; the Angular CLI compiles it to JavaScript for you.

b. Diagram



TypeScript is checked and compiled to JavaScript

c. Syntax

```ts
let name: string = 'Angular';
let count: number = 10;
let tags: string[] = ['a', 'b'];

interface User { id: number; name: string; email?: string }  // ? = optional

function add(a: number, b: number): number { return a + b; }

class Student { constructor(public name: string) {} }

function first<T>(items: T[]): T | undefined { return items[0]; }  // generic
```

d. Example Program

`Programs/03_TypeScript_Essentials/types-basics.ts`

```ts
// Run with: npx tsx types-basics.ts
export {};

// 1. Basic types
const courseName: string = 'Angular';
let lessons: number = 38;
const isFun: boolean = true;
const topics: string[] = ['Components', 'Services', 'Routing'];
const level: [string, number] = ['Beginner', 10]; // tuple: fixed length and types

// 2. Union of literal values: only these three strings are allowed
type Level = 'Beginner' | 'Intermediate' | 'Advanced';

function describe(level: Level, count: number = 1): string {
  return `${count} topic(s) at ${level} level`;
```

```
}

// 3. Generic function: works for any type T
function first<T>(items: T[]): T | undefined {
  return items[0];
}

console.log(`${courseName} has ${lessons} lessons. Fun? ${isFun}`);
console.log('Topics:', topics.join(', '));
console.log('Tuple:', level[0], level[1]);
console.log(describe('Beginner', 10));
console.log('First topic:', first(topics));
console.log('First of empty list:', first<number>([]));

// describe('Expert'); // <- compile error: 'Expert' is not a valid Level
```

More programs in the same folder: `classes-interfaces.ts`

e. Output

Console output (run with `npx tsx`):

```
Angular has 38 lessons. Fun? true
Topics: Components, Services, Routing
Tuple: Beginner 10
10 topic(s) at Beginner level
First topic: Components
First of empty list: undefined
```

f. Real-World Example in Tamil (தமிழ் உதாரணம்)

ரேஷன் கடையில் பொருள் வாங்கும்போது அரிசிக்கு தனி அளவு (கிலோ), எண்ணெய்க்கு தனி அளவு (லிட்டர்) என்று ஒவ்வொன்றுக்கும் ஒரு வகை உண்டு. யாராவது 'இரண்டு லிட்டர் அரிசி' என்று கேட்டால் கடைக்காரர் உடனே 'அது தவறு' என்று சொல்லிவிடுவார்.

TypeScript-உம் அப்படியே. `number`, `string` என்று ஒவ்வொரு மதிப்புக்கும் வகையைச் சொல்லிவிட்டால், தவறான வகையைப் பயன்படுத்தினால் நீங்கள் program-ஐ ஓட்டுவதற்கு முன்பே editor எச்சரிக்கை கொடுக்கும். இதனால் பிழைகள் குறையும்.

g. Common Mistakes

- Using `any` everywhere. It switches off type checking and defeats the purpose of TypeScript.
- Forgetting that types disappear at runtime; you cannot check an interface with `typeof` or `instanceof`.
- Mixing `==` and `===`; always prefer `===`.
- Ignoring the editor's red underline and compiling anyway.

h. Interview Questions

Q1. What is TypeScript and why does Angular use it?
A typed superset of JavaScript. Types catch bugs early, give great editor support and make large apps easier to maintain.

Q2. Interface vs type alias?
Both describe shapes. Interfaces can be extended/merged and suit object shapes; type aliases can also name unions, tuples and primitives.

Q3. What is `any` vs `unknown`?
`any` turns off checks. `unknown` is a safe version: you must narrow the type before using the value.

Q4. What are generics?
Type parameters (like `T`) that let functions and classes work with many types while staying type-safe.

Q5. What does `?` after a property name mean?
The property is optional; its type is `T | undefined`.

i. Practice Exercises

1. Create an interface `Product` (id, name, price, inStock?) and an array of three products; print the names of those in stock.
2. Write a generic function `last<T>(items: T[])` that returns the last item, and test it with numbers and strings.
3. Make a class `BankAccount` with a private balance, `deposit`, `withdraw` (no negative balance) and a `balance` getter.